Naan Mudhalvan Project Report

AUTO TICKET CLASSIFICATION USING FLOW DESIGNER

Submitted by

J CHRISH MARIYA THERASHA
M NOORUL HASINA SHRIN
S PREETHI RENO
R GOBIKA

Under the Guidance of

Dr. Devi Shalini K B
MCA MPhil Ph.D

In partial fulfilment for the award of the degree of

BACHELOR OF SCIENCE IN ARTIFICIAL INTELLIGENCE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

MARIA ARTS AND SCIENCE COLLEGE FOR WOMEN

(Affiliated to Manonmaniam Sundaranar University, Tirunelveli)
TIRUCHENDUR ROAD, VALLIOOR – 627 117, Tamil Nadu, India.

OCTOBER – 2026

1. EXECUTIVE SUMMARY

The Auto Ticket Classification using Flow Designer project is a ServiceNow-based IT helpdesk automation solution. The project addresses the repetitive task of manually identifying the type of an incident and selecting the appropriate Category and Subcategory. The proposed solution uses Flow Designer to evaluate information entered in the Short Description and Description fields and apply predefined classification rules.

The project is designed around common educational IT support scenarios such as Wi-Fi connectivity issues, projector failures, password problems, slow computers, and email access issues. By using a rule-based workflow, the project provides a transparent and maintainable approach to initial ticket classification.

The documentation covers the complete project lifecycle including problem identification, objectives, requirements, system design, development, testing, acceptance, results, limitations, maintenance, and conclusion. It also describes how the solution can be extended in future versions.

The project demonstrates practical skills in ServiceNow Incident Management, Flow Designer, field configuration, conditions, actions, testing, troubleshooting, and technical documentation.

1.1 Key Project Outcome

Area	Outcome
Platform	ServiceNow
Automation	Flow Designer
Input	Short Description and Description
Processing	Keyword and condition based rules
Output	Category and Subcategory
Validation	Functional and acceptance testing
Target Environment	School / College IT Helpdesk

2. INTRODUCTION

Information technology support is an important part of modern educational institutions. Students, teachers, administrative staff, and other users may raise incidents when they experience problems with network connectivity, computers, projectors, passwords, email, or other IT services.

In a conventional helpdesk process, every incident is reviewed by a support person. The agent reads the request, understands the issue, chooses a category, selects a subcategory, and then continues with resolution. When the number of incidents increases, this classification activity can become repetitive and time-consuming.

The Auto Ticket Classification using Flow Designer project introduces automation at this initial classification stage. Instead of requiring the agent to manually classify every common incident, the ServiceNow flow checks configured information and updates the incident according to predefined rules.

The solution is intentionally designed as a rule-based system so that the project team can understand, demonstrate, test, and maintain the automation. It provides a foundation for future improvements without making the initial implementation unnecessarily complex.

2.1 Documentation Purpose

This project report records the functional idea, technical approach, implementation process, test strategy, acceptance criteria, results, and future enhancement opportunities. The document can be used as evidence of project development and as a reference for future maintenance.

3. BACKGROUND AND PROBLEM STATEMENT

3.1 Background

ServiceNow provides a structured platform for managing incidents and automating IT service processes. Incident records contain useful information such as the caller, short description, detailed description, impact, urgency, priority, assignment group, category, and subcategory. Proper classification helps support teams organize incidents and maintain useful service data.

3.2 Existing Process

In the manual process, the user or support agent enters the incident description. The support agent then identifies the issue type and manually selects the classification. The quality of the classification depends on the person handling the ticket and the consistency of the classification rules being followed.

3.3 Problem Statement

Manual classification of repetitive IT incidents can create additional workload and may lead to inconsistent classification for similar requests. A simple automated solution is therefore required to identify common incident types from the information entered by the user and assign suitable Category and Subcategory values.

3.4 Proposed Solution

The proposed solution uses ServiceNow Flow Designer. When a relevant incident event occurs, the flow evaluates configured conditions. If the incident contains information matching a supported rule, the flow updates the classification fields automatically.

4. PROJECT AIM, OBJECTIVES AND SCOPE

4.1 Project Aim

The main aim is to develop a ServiceNow Flow Designer automation that classifies common IT helpdesk incidents using predefined rules and reduces repetitive manual classification effort.

4.2 Project Objectives

• Identify common IT helpdesk incident types.
• Define suitable Category and Subcategory rules.
• Analyze Short Description and Description information.
• Build the classification automation using Flow Designer.
• Test both matching and non-matching incidents.
• Verify the automation through project acceptance testing.
• Document the complete development and testing process.

4.3 Scope

Included	Not Included / Future
Common IT incident classification	Advanced AI/ML classification
Keyword and condition rules	External helpdesk integrations
Category/Subcategory updates	Complex predictive analytics
Flow Designer implementation	Large-scale production rollout
Testing and documentation	Natural-language model training

4.4 Target Users

Target users include IT helpdesk agents, ServiceNow administrators, support staff, project team members, and users who create IT incident requests.

5. REQUIREMENT ANALYSIS

5.1 Functional Requirements

ID	Requirement	Expected Behaviour
FR-01	Incident creation	User or agent can create an incident record.
FR-02	Input analysis	Short Description and Description can be evaluated.
FR-03	Rule matching	Configured conditions identify supported incident types.
FR-04	Category update	Correct Category is assigned for a matching rule.
FR-05	Subcategory update	Correct Subcategory is assigned where applicable.
FR-06	Flow execution	Flow runs when trigger conditions are met.
FR-07	Unmatched handling	Unmatched requests are not given unrelated classifications.
FR-08	Testing	Configured behaviour can be validated using test incidents.

5.2 Requirement Priority

Priority	Meaning	Examples
High	Essential for project operation	Trigger, classification, category update
Medium	Important supporting functionality	Rule maintenance, validation
Low	Future improvement	Advanced intelligence and reporting

6. NON-FUNCTIONAL REQUIREMENTS AND USE CASES

6.1 Non-Functional Requirements

• **Usability:** Supported incidents should require minimal manual classification effort.
• **Consistency:** Similar descriptions should follow the same configured rules.
• **Maintainability:** Rules should be easy to understand and update.
• **Reliability:** The flow should execute according to its trigger and conditions.
• **Security:** Configuration changes should be limited to authorized users.
• **Traceability:** Project changes and test results should be documented.

6.2 Main Use Cases

Use Case	Actor	Process
Create Incident	End User / Agent	Enter incident information and submit.
Evaluate Incident	Flow Designer	Check trigger and configured conditions.
Classify Incident	Flow Designer	Match the incident to a classification rule.
Update Record	Flow Designer	Set Category and Subcategory.
Review Incident	Helpdesk Agent	Verify classification and continue support.
Maintain Rules	Administrator	Modify rules when requirements change.

6.3 User Perspective

From the end-user perspective, the process remains simple: the user describes the problem in the incident form. The automation works in the background and provides structured classification for supported requests.

7. SYSTEM OVERVIEW AND ARCHITECTURE

7.1 System Overview

The system consists of a ServiceNow Incident record and a Flow Designer automation. The incident acts as the input record. Flow Designer provides the processing logic. The Category and Subcategory fields represent the structured output of the classification process.

7.2 High-Level Architecture

User / Agent → Incident Form → Incident Record → Flow Designer Trigger → Conditions / Rules → Update Record Action → Category & Subcategory → Classified Incident

7.3 Architecture Components

Component	Role
Incident Form	Collects the user's IT problem information.
Short Description	Provides concise incident information.
Description	Provides additional context for classification.
Flow Trigger	Determines when the automation starts.
Conditions	Check configured incident information.
Actions	Update the incident fields.
Category	Stores broad classification.
Subcategory	Stores detailed classification.

7.4 Data Flow

Incident information enters the system first. The flow then evaluates the configured rule conditions. If a rule matches, the corresponding classification is written back to the incident. The updated record can then be used by the helpdesk for further processing and reporting.

8. DATA MODEL AND CLASSIFICATION RULES

8.1 Incident Data Used

The project primarily uses the Short Description and Description fields as textual inputs. The output of the classification process is represented through the Category and Subcategory fields. Other incident fields may remain available for normal ServiceNow incident processing.

8.2 Example Classification Matrix

Incident Type	Example Keywords	Category	Subcategory
Wi-Fi Issue	Wi-Fi, wireless, connection	Network	Wi-Fi / Network
Projector Issue	projector, display	Hardware	Projector
Password Issue	password, reset password	Software	Password
Slow Computer	slow computer, system slow	Hardware	Computer
Email Issue	email, mailbox, email access	Software	Email
Unmatched	No configured keyword	Default / unchanged	Default / unchanged

8.3 Rule Design Principles

Rules should be clear, specific, testable, and easy to maintain. The project should avoid assigning a classification merely because an unrelated word happens to appear in a description. Testing should therefore include both realistic matching examples and negative examples.

8.4 Extensibility

New rules can be introduced for printer problems, software installation, account access, VPN problems, classroom devices, or other frequently reported issues. Each new rule should be documented and tested before being used in the project demonstration.

9. DEVELOPMENT PLAN AND IMPLEMENTATION

9.1 Development Stages

Stage	Activity	Deliverable
1	Requirement analysis	Requirement list
2	Environment preparation	ServiceNow development setup
3	Data configuration	Category/Subcategory configuration
4	Rule planning	Classification rule matrix
5	Flow development	Configured Flow Designer flow
6	Testing	Test cases and results
7	Correction	Updated configuration
8	Documentation	Final project report

9.2 Implementation Approach

The implementation begins by preparing the ServiceNow environment and identifying the incident fields required by the project. Classification choices are reviewed before configuring the flow. The Flow Designer automation is then created with an appropriate trigger, conditions, and update actions.

9.3 Configuration Validation

Each configuration stage is validated independently. The project team verifies that field values are available, conditions are logically correct, actions update the intended record, and the flow can execute without configuration errors.

9.4 Version and Change Control

Development changes should be recorded through appropriate ServiceNow development practices such as update sets where applicable. This helps the team track configuration changes and maintain a clear project history.

If incident description contains Wi-Fi or wireless terms → **Then** assign Network / Wi-Fi classification.
If description contains projector terms → **Then** assign Hardware / Projector.
If description contains password terms → **Then** assign Software / Password.
If description contains slow computer terms → **Then** assign Hardware / Computer.
If description contains email access terms → **Then** assign Software / Email.

10.5 Flow Maintenance

When a rule is changed, the affected test cases should be executed again. This helps ensure that a new rule does not unintentionally change the classification of existing incident types.

11. USER INTERFACE AND USER EXPERIENCE

11.1 Incident Creation Experience

The user should be able to create an incident using the standard ServiceNow incident interface. The user enters a meaningful Short Description and, where necessary, additional details in the Description field. The automation should not require the user to understand the internal Flow Designer logic.

11.2 Helpdesk Agent Experience

When an incident is classified automatically, the helpdesk agent can review the Category and Subcategory and continue with normal incident handling. If the classification is not appropriate, the agent can follow the organization's correction process.

11.3 Usability Considerations

The project aims to keep the user experience simple. The classification logic works in the background. Clear incident descriptions improve the likelihood of correct rule matching, so users should be encouraged to provide useful problem descriptions.

11.4 Data Quality

Good classification depends on meaningful input. Incomplete or ambiguous descriptions may not match any rule. The project therefore treats input quality as an important supporting factor in successful automation.

11.5 Accessibility and Clarity

The workflow does not introduce a separate complex interface for the user. It works with the existing incident process, which helps keep the project understandable for both end users and helpdesk personnel.

12. TESTING STRATEGY

12.1 Testing Objective

The objective of testing is to verify that the Flow Designer automation produces the expected classification for supported incidents and behaves safely when a request does not match a configured rule.

12.2 Testing Levels

Testing Level	Purpose
Configuration Testing	Verify individual flow components and field settings.
Functional Testing	Verify classification behaviour for supported scenarios.
Negative Testing	Verify unmatched incidents are not incorrectly classified.
Regression Testing	Verify changes do not break existing successful scenarios.
Acceptance Testing	Verify the project meets defined acceptance criteria.

12.3 Test Data

Representative test data includes Wi-Fi problems, projector failures, password requests, slow computer complaints, email access issues, and unrelated requests. Test descriptions should be realistic enough to demonstrate the behaviour expected in an educational IT helpdesk.

12.4 Expected Test Result

A test is successful when the actual Category and Subcategory values match the expected values and the flow execution completes according to the configured design.

12.5 Test Preparation

Before testing, the project team should confirm that the development environment is available, the flow is in the intended state, test data is ready, and the expected classification for each test case has been documented.

13. TEST CASES AND RESULTS

13.1 Functional Test Cases

ID	Test Input	Expected Classification	Status
TC-01	Wi-Fi is not connecting	Network / Wi-Fi	PASS after verification
TC-02	Classroom projector is not working	Hardware / Projector	PASS after verification
TC-03	Unable to reset password	Software / Password	PASS after verification
TC-04	Computer is running slowly	Hardware / Computer	PASS after verification
TC-05	Unable to access email	Software / Email	PASS after verification
TC-06	Unrelated timetable request	No inappropriate IT classification	PASS after verification

13.2 Negative Testing

Negative testing is important because the automation should not classify every incident simply because it contains a general word. An unrelated request should not receive a random IT classification. If no configured condition is satisfied, the flow should follow the intended default behaviour.

13.3 Regression Testing

Whenever an existing rule is modified or a new rule is added, previously successful test cases should be repeated. This confirms that the new configuration does not produce unintended changes in existing classifications.

13.4 Test Evidence

For final project submission, screenshots of the incident input, flow configuration, flow execution, and final classification can be attached as supporting evidence where required.

13.5 Result Recording

Each test should record the test ID, input data, expected result, actual result, status, and any corrective action. This provides a clear record for project review and demonstration.

14. PROJECT ACCEPTANCE TEST

14.1 Acceptance Purpose

Project Acceptance Testing confirms that the completed solution satisfies the project requirements from an end-to-end perspective. It is not limited to checking individual configuration components; it verifies the complete user-to-classification process.

14.2 Acceptance Criteria

Criterion	Acceptance Requirement
AC-01	Incident can be created successfully.
AC-02	Flow trigger executes according to configuration.
AC-03	Supported keywords/conditions are detected.
AC-04	Correct Category is assigned.
AC-05	Correct Subcategory is assigned where applicable.
AC-06	Unmatched requests do not receive an inappropriate classification.
AC-07	Corrections can be retested successfully.
AC-08	Documentation is complete for project demonstration.

14.3 Acceptance Status

The project is ready for acceptance when the defined test cases have been executed, expected results have been confirmed, identified defects have been corrected and retested, and the required documentation has been completed.

14.4 Acceptance Evidence

Useful evidence includes successful incident records, screenshots of flow configuration, flow execution details, classification results, and a completed acceptance checklist.

15. DEFECT HANDLING AND TROUBLESHOOTING

15.1 Common Defect Types

Defect	Possible Cause	Corrective Action
Flow does not run	Incorrect trigger or conditions	Review trigger and condition configuration.
Wrong Category	Incorrect rule mapping	Review classification matrix and action.
Wrong Subcategory	Invalid choice or mapping	Verify field choices and action values.
Unmatched ticket	Missing keyword/rule	Add or refine a suitable condition.
Unexpected classification	Overly broad condition	Make the condition more specific.
Update failure	Invalid field value	Verify configured choice values.

15.2 Troubleshooting Method

When a defect is identified, first reproduce the issue using the same test input. Next, review the flow trigger and conditions. Then inspect the action responsible for updating the incident. After correcting the configuration, execute the same test again and perform regression testing where required.

15.3 Defect Documentation

Each significant defect should record the test case, observed result, expected result, probable cause, correction made, and retest result. This provides traceability and supports future maintenance.

15.4 Retesting

A corrected test case should be executed again under the same conditions. If the result matches the expected outcome, the defect can be marked resolved. Related test cases should also be reviewed when the correction affects shared conditions.

16. RESULTS, BENEFITS AND PROJECT VALUE

16.1 Results

The project demonstrates that common IT helpdesk requests can be processed through a structured ServiceNow Flow Designer automation. Supported incident types can be mapped to predefined classification values, providing a consistent starting point for helpdesk processing.

16.2 Benefits

- Reduces repetitive manual classification effort.
- Improves consistency for common incident types.
- Provides structured incident data for reporting.
- Makes the classification process easier to demonstrate and understand.
- Provides a maintainable rule-based foundation.
- Supports learning of ServiceNow automation concepts.

16.3 Project Value

The project provides both practical and educational value. From a technical perspective, it demonstrates how a business requirement can be converted into a ServiceNow workflow. From a learning perspective, it connects incident management, data configuration, automation, testing, and documentation into one complete project.

16.4 Expected Operational Impact

In a suitable helpdesk environment, automatic initial classification can help agents start with a structured incident record. The actual operational impact would depend on the quality of rules, incident volume, input quality, and the organization's ServiceNow configuration.

17. LIMITATIONS, RISKS AND FUTURE ENHANCEMENTS

17.1 Limitations

The current implementation is based on predefined keywords and conditions. It may not understand unusual phrasing, spelling variations, multiple issues in one incident, or descriptions that do not match the configured rules. It does not provide advanced semantic or machine-learning classification.

17.2 Risk Management

Risk	Impact	Mitigation
Incomplete rules	Some incidents remain unclassified	Review incident patterns and add rules.
Broad keywords	Incorrect classification	Use specific conditions and test negative cases.
Configuration change	Existing behaviour may change	Use documentation and regression testing.
Poor input quality	Rule matching may fail	Encourage meaningful incident descriptions.
Flow error	Automation may stop	Monitor execution and troubleshoot configuration.

17.3 Future Enhancements

Future versions may include more classification rules, richer condition logic, notifications, dashboards, reporting, automated assignment groups, knowledge article suggestions, and AI/ML-assisted classification. Any such enhancement should be introduced through a controlled requirements and testing process.

17.4 Scalability Consideration

If the number of classification rules grows significantly, the rule structure should be reviewed to avoid overlapping or conflicting conditions. A documented classification matrix and controlled change process can help maintain predictable behaviour.

18. PROJECT MANAGEMENT, MAINTENANCE AND DOCUMENTATION

18.1 Work Breakdown Structure

No.	Work Package	Deliverable
1	Requirement Analysis	Business use case and requirements
2	Environment Setup	ServiceNow project environment
3	Data & Rule Planning	Classification matrix
4	Flow Development	Working automation
5	Testing	Test cases and results
6	Acceptance	Acceptance evidence
7	Documentation	Final project report
8	Demonstration	Project demo and explanation

18.2 Maintenance Plan

The classification rules should be reviewed periodically. New incident patterns should be analyzed to identify missing rules. Changes should be tested before they are used in the project environment. Existing test cases should be retained as a regression test suite.

18.3 Documentation Standards

Technical documentation should describe the purpose of the flow, trigger conditions, classification rules, update actions, test cases, known limitations, and maintenance procedure. Screenshots can be used to support important configuration evidence.

18.4 Project Deliverables

Major deliverables include the requirement analysis, design information, configured Flow Designer automation, classification rule set, test results, acceptance test, project documentation, and final demonstration materials.

18.5 Team Responsibilities

The team should divide responsibilities across requirement analysis, ServiceNow configuration, flow development, testing, documentation, and demonstration. A clear division of work supports timely completion and reduces duplication.

19. CONCLUSION

The **Auto Ticket Classification using Flow Designer** project demonstrates a practical approach to automating the initial classification of IT helpdesk incidents in ServiceNow. By analyzing incident descriptions and applying predefined rules, the solution can assign appropriate Category and Subcategory values for supported request types.

The project follows a complete development lifecycle: identifying the problem, defining objectives, analyzing requirements, designing the solution, configuring the ServiceNow workflow, testing the automation, performing project acceptance testing, documenting the results, and identifying future improvements.

The use of Flow Designer provides a visual and maintainable method for implementing the automation. The rule-based approach is transparent, making it suitable for demonstration and easy for administrators to understand and modify. At the same time, the project recognizes that rule-based classification has limitations and can be extended in future versions.

The testing and acceptance activities provide a structured method for verifying that the automation works for common scenarios and does not create inappropriate classifications for unmatched requests. This supports the overall reliability and maintainability of the project.

19.1 Final Outcome

Project Area	Final Outcome
Problem	Manual classification of common IT incidents
Solution	Automated rule-based classification
Platform	ServiceNow
Automation Tool	Flow Designer
Input	Short Description and Description
Output	Category and Subcategory
Validation	Functional, negative, regression and acceptance testing
Documentation	Complete 20-page project report
Project Status	Ready for demonstration and submission

19.2 Final Statement

Overall, the project provides a clear and practical demonstration of ServiceNow workflow automation for an educational IT helpdesk. It reduces repetitive classification work, promotes consistent incident data, and provides a strong foundation for future enhancements such as richer rules, reporting, notifications, and intelligent classification.

END OF DOCUMENTATION PHASE PROJECT REPORT